GRADUATE COURSE · 14 WEEKS · PYTHON

Advanced Data Analysis

For the social sciences

Week 1 — Introduction

M. Fuat Kına

fuatkina@gmail.com · mkina@ku.edu.tr

LEARNING OUTCOMES

What you will be able to do by week 14

Program

Write and debug idiomatic Python for data-intensive research tasks.

Collect

Acquire, clean, merge and reshape data from files, APIs and web sources.

Predict

Design a supervised pipeline and evaluate it with the right metric.

Identify

Tell predictive from causal questions, and use ML methods built for estimation.

Measure

Use language models as instruments, validated against human-coded ground truth.

Publish

Produce a documented, reproducible project in a public GitHub repository.

THE TOOL

Why Python?

- Readable code — beginner friendly, yet powerful
- The default language of scientific computing
- One ecosystem for the whole pipeline: pandas, scikit-learn, PyTorch, Hugging Face
- Free and open source; enormous community and documentation
- Everything at the LLM frontier ships in Python first

Alternatives

R · Julia

STATA · SPSS

QGIS · ArcGIS

No prior programming experience is assumed. Curiosity, patience and a willingness to debug are.

STRUCTURE

Five movements, fourteen weeks

I · WEEKS 1–3

Programming Foundations

Python core, functions and loops,
NumPy and Matplotlib

II · WEEKS 4–6

Data: Wrangling & Collection

pandas end to end;
web scraping and APIs

III · WEEKS 7–10

Machine Learning for Research

Models, evaluation, tuning;
causal inference with ML

IV · WEEKS 11–12

LLMs in the Research Pipeline

Annotation and distillation;
synthetic agents and ethics

Module V (weeks 13–14): an elective session set collectively by the class, then final presentations.

MODULE IV

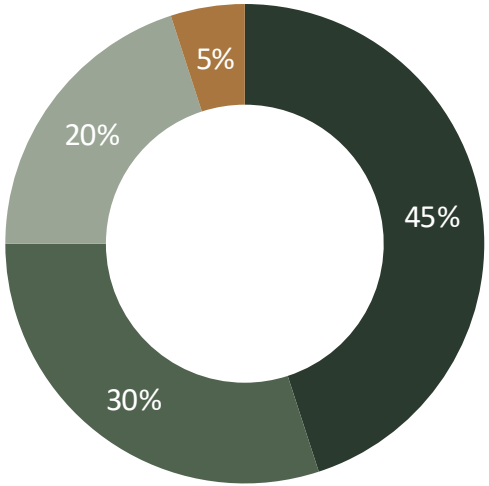
The part that is not in a standard curriculum

Language models enter the research pipeline as measurement instruments. Two sessions, four practical questions:

Can a model annotate?	Prompting as an operationalisation problem: codebooks, few-shot design, structured output — then validation against a human-coded gold standard.
Can it be made small?	Teacher–student distillation: silver labels, noise diagnostics, a BERT-family student you can freeze and re-run in five years.
Can it replace a respondent?	Persona conditioning and algorithmic fidelity; calibration against survey microdata; where it quietly breaks.
Should it?	Flattened variance, temporal drift, minority positions misrepresented — and the ethics of replacing people with plausible text.

ASSESSMENT

Grading



45 % — **Assignments (5)**

A4, the machine-learning shared task, is out of 13 points; the other four out of 8.

30 % — **Final project**

Presentation, ~2,000-word report, the code, and a GitHub repo. Link only.

20 % — **Attendance & participation**

Includes one in-class paper presentation per student.

5 % — **Midterm report**

One page maximum. A research proposal in miniature.

DELIVERABLES

Five assignments

1	**Programming fundamentals**	released week 3
2	**Data wrangling with Pandas**	released week 5
3	**Web scraping and APIs**	released week 6
4	**Machine learning**	released week 8
5	**LLM annotation & distillation**	released week 11

Two of them carry the course

A4 — a shared task. Build the full pipeline: preprocessing, model selection, training and testing, hyperparameter tuning, reported outputs. Higher scores are rewarded.

A5 — annotate a small corpus with a language model, validate it against a human-coded gold standard, and distil the labels into a supervised classifier.

Readability counts: comment your code where a reader would otherwise have to guess.

YOUR OWN RESEARCH

Midterm report & final project

Midterm report · 1 page · week 6

- Define a research purpose
- Describe methods that might work
- List the sources you may use
- Explain how you will access the data

A research proposal in miniature — be precise.

Final project · 30 % · week 14

- Class presentation
- Written report, ~2,000 words
- The Python code you wrote
- A GitHub repo holding all of it — you submit the link only

Times New Roman, 12 pt, single-spaced, PDF.

Generate data that does not exist yet, explain something in your own field, or build a synthetic sample and run an experiment inside it — as long as it is Python.

REPRODUCIBILITY

Your repository is your submission

1

Create

A public repo named “Python_Course”

→

2

Invite

Add the instructor and the TA as collaborators

→

3

Commit

Each week: scripts and reports (PDF), named clearly

→

4

Submit

For the final project, you send only the link

Never used Git? It takes an afternoon. A commit history is not bureaucracy — it is what makes your findings checkable.

Reading for your GitHub page: Gandrud, C. (2013). SSRN 2199367.

CORE MATERIALS

What to keep on your desk

VanderPlas (2016)

Python Data Science Handbook — the main technical reference (ch. 2–5). Open access.
`jakevdp.github.io/PythonDataScienceHandbook`

Shaw (2017)

Learn Python 3 the Hard Way — exercises 0–40 across the programming weeks.
`learnpythonthehardway.org/python3`

Grimmer, Roberts & Stewart (2022)

Text as Data — the framework behind Modules III and IV.

Jurafsky & Martin (3rd ed. draft)

Speech and Language Processing — reference chapters on transformers and LLMs.
`web.stanford.edu/~jurafsky/slp3`

Official Python documentation

The habit worth building: read the docs before the blog post.
`docs.python.org/3`

Weekly readings are mandatory unless marked “optional”. Beginners: w3schools.com/python is a gentle start.

THE WORKBENCH

Where will you write code?

Google Colab

NOTHING TO INSTALL

- Runs in the browser
- Free GPU — we need it in Module IV
- Easiest way to share a notebook
- Needs a connection

Jupyter Notebook / Lab

EXPLORING & REPORTING

- Cell-by-cell exploration
- Code, output and prose together
- Ships with Anaconda
- Awkward for larger projects

VS Code

WRITING REAL PROJECTS

- Editor, terminal and Git in one window
- Runs .py and .ipynb alike
- Debugger, linting, extensions
- Slightly steeper start

Rule of thumb: explore in a notebook, ship from VS Code, borrow Colab's GPU when you need one.

INSTALLATION

conda, miniconda, anaconda

`conda`	A package and environment manager. Open source. It is the engine.
`Miniconda`	A minimal installer: conda plus Python, nothing else. You add what you need.
`Anaconda`	The full distribution: conda, Python, ~250 scientific packages, Navigator, Jupyter.

For this course, install Anaconda — it gives you Jupyter and most of what we need on day one.

INSTALLATION

Environments — and why you want them

One project may need NumPy 1.26 while another still depends on 1.19. An environment keeps those worlds apart, so installing something for today's analysis cannot silently break last month's.

In VS Code, pick the environment from the interpreter selector in the status bar (bottom-right); the same list appears when you open a notebook.

```
conda create -n ada python=3.11
conda activate ada
conda install pandas numpy matplotlib
pip install scikit-learn transformers
conda env list
conda deactivate
```

docs.conda.io → user-guide → tasks → manage-environments

BEFORE WEEK 2

Setting up, step by step

01 **Install Anaconda**

docs.anaconda.com/anaconda/install — accept the defaults.

02 **Open Jupyter Lab**

From Anaconda Navigator, or type jupyter lab in the terminal.

03 **Install VS Code**

Add the Python and Jupyter extensions from the marketplace.

04 **Select your interpreter**

Ctrl/Cmd+Shift+P → “Python: Select Interpreter” → your conda env.

05 **Try Google Colab**

colab.research.google.com — open our week-1 notebook there.

06 **Create your GitHub repo**

…and add the instructor and the TA as collaborators.

Installation guidelines are on the course GitHub page.

Before we meet again

Install

Anaconda, VS Code, and a GitHub account.

Read

Lazer et al. (2009), “Computational Social Science”; Edelmann et al. (2020), “Computational Social Science and Sociology”.

Browse

w3schools.com/python — data types, lists, dictionaries.

Decide

Which paper you would like to present this term.